

ParcelSafe - Secure Delivery Locker System

MINI PROJECT REPORT

Embedded Systems Design Project

EEE2301



Group Members:

1. Siluni Pasqual – 22UG2-0145
2. Sasindu Basnayake – 22UG2-0215

Faculty of Engineering
SLTC Research University

AUGUST 2024

Table of Contents

CHAPTER 01 : INTRODUCTION	3
1.1 BACKGROUND OF THE PROJECT.....	3
1.2 FEATURES.....	3
CHAPTER 02 : LITERATURE SURVEY	4
2.1 REVIEW OF THE EXISTING LITERATURE.....	4
CHAPTER 03 : METHODOLOGY	5
3.1 BLOCK DIAGRAM AND DEVICE ARCHITECTURE.....	5
3.2 COMPONENT SELECTION AND CIRCUIT DESIGN	5
3.3 SIMULATED CIRCUIT DIAGRAM.....	7
3.4 SCHEMATIC DESIGN AND PCB LAYOUT	8
3.4.1 Schematic Design.....	8
3.4.2 PCB Layout Design	9
3.4.3 PCB Manufacturing	10
3.5 ENCLOSURE DESIGN	10
3.5.1 SOLIDWORKS Design.....	10
3.5.2 Enclosure Manufacturing Methods.....	11
3.6 TOOLS, TECHNOLOGIES AND FRAMEWORKS EMPLOYED	12
CHAPTER 04 : RESULTS AND ANALYSIS.....	13
4.1 OUTCOMES AND RESULTS OF THE PROJECT	13
4.1.1 System Operation Workflow.....	13
4.2 COMPARISON OF THE RESULTS WITH THE EXISTING LITERATURE	15
4.3 WIFI CONNECTIVITY	15
CHAPTER 05 : CONCLUSION AND FUTURE WORK	17
5.1 FUTURE RESEARCH OR IMPROVEMENTS	17
5.2 CONCLUSION.....	18
5.3 WORKPLAN	18
5.3.1 Project Timeline	18
5.3.2 Bill of Materials	19
5.3.3 Proposed Product Price	19
APPENDICES – PROGRAM CODE.....	20
REFERENCES	34

CHAPTER 01 : INTRODUCTION

1.1 BACKGROUND OF THE PROJECT

In today's fast-paced world, the rise in online shopping has led to increased home deliveries, but it also poses challenges like theft and missed deliveries when no one is home. ParcelSafe addresses this by offering a Secure Delivery Locker System. It provides a sturdy, weatherproof locker that ensures the safe delivery of packages even when the recipient is absent. Delivery personnel use a keypad to unlock the locker and deposit the parcel, while the recipient receives notifications via the Telegram app, allowing them to retrieve their package at their convenience.

1.2 FEATURES

- An Infrared (IR) sensor is integrated to detect the presence of a parcel upon door opening, activating a buzzer to indicate access and ensure the parcel is securely deposited inside the locker.
- A keypad entry system allows delivery personnel to enter a provided code. Only if '*' is pressed will it ask the delivery personnel to enter the code. An LCD display shows messages such as "Enter Code," "Please wait," and "Granted."
- Each delivery personnel are provided with a same single code.
- A notification system is integrated to inform recipients in real-time upon parcel delivery via the Telegram app. Once the delivery personnel come to deliver the parcel, he should enter that code, and if he enters the correct code, the recipient is notified with a Telegram message whether to unlock the box. The box can be unlocked only if the user grants access via the Telegram App even if the correct code is entered, ensuring security of the box.
- The system is powered by a two separate 3.7V Li-ion rechargeable batteries, one to give power to the PIC, LCD, IR sensor and the ESP8266 Wi-Fi module, and the other to powerup the lock, with two boost converters (providing 5V and 10V outputs) for efficient power management.
- The user specified Wi-Fi SSID and its corresponding password are stored in the EEPROM of the ESP8266.
- A master password is provided with the manual, which can change the delivery code, which is provided to the delivery personnels, by entering this password. This password is only known by the user of the box.

CHAPTER 02 : LITERATURE SURVEY

2.1 REVIEW OF THE EXISTING LITERATURE

1. Parcel Locker Networks

Overview and Functionality

Parcel locker networks consist of secure, automated series of postal boxes located in accessible central areas to facilitate package delivery and collection. These systems aim to simplify last-mile delivery by providing a flexible and convenient option for customers.

Applications

- **Central Areas:** Apartment complexes, office buildings, and business centers.
- **Educational Institutions:** Campuses and dormitories.
- **Public locations:** Parks, central business districts, and libraries.
- **Logistics:** Warehouses and distribution centers.

Case Study: [Cainiao Smart Lockers in China](#)

2. Vpod Automated Parcel Lockers

Overview and Functionality

The Vpod Automated Parcel Delivery Locker System automates the package delivery process, minimizing manual labor and enhancing security and convenience.

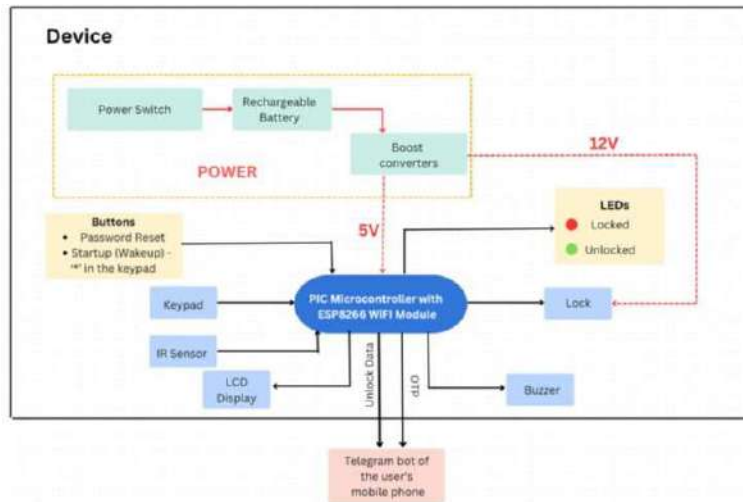
Applications

- **Central Areas:** Apartments, offices and business centers.
- **Healthcare:** Pharmacies and prescription deliveries.
- **Hospitality:** Hotels and other accommodation facilities.
- **Educational Institutions:** Campuses and schools for secure deliveries.

Case Study: [Vpod in Office Buildings](#)

CHAPTER 03 : METHODOLOGY

3.1 BLOCK DIAGRAM AND DEVICE ARCHITECTURE



3.2 COMPONENT SELECTION AND CIRCUIT DESIGN

❖ Wi-Fi Module

The ESP8266 Wi-Fi module was selected for its reliable connectivity, compatibility with the PIC16F877A microcontroller, and cost-effectiveness. Its compact size and strong community support make it ideal for real-time communication with the Telegram bot, enabling remote monitoring and notifications. Wi-Fi was chosen over Bluetooth for its superior range, essential for long-distance connectivity.



Figure: ESP8266 Wi-Fi Module

❖ Lock

Two main electronically controlled lock designs were considered,

- Solenoid lock
- Servo motor attached to a locking mechanism



Figure: Solenoid Lock

The servo motor-based design was rejected due to its low strength, limited durability, and higher cost. Instead, a solenoid lock was chosen for its greater strength and cost-effectiveness. Since the solenoid lock requires 10V-12V, a 5V relay was used to control it from the microcontroller, ensuring isolation between the low-power control circuit and the high-power lock circuit.

❖ Keypad

A 4x3 matrix array keypad was chosen to enable users to input numbers and to provide "ENTER" function.



Figure: Keypad

❖ LED, Buzzer and Switch

An LED and a buzzer are used as indicators. A small push button switch is used to reset Wi-Fi.

❖ Sensors

Two options were considered when selecting a sensor to detect the door's status:

- A force sensor
- An IR sensor

After evaluating both options, the force sensor was found to have limitations: it can be easily damaged and may not reliably detect small, lightweight parcels such as documents. Additionally, there is a security concern where a thief could potentially manipulate the sensor by placing a weight to deceive it into registering an incorrect status. Therefore, the IR sensor was chosen for its reliability, durability, and ability to accurately detect whether the door is open or closed.

3.3 SIMULATED CIRCUIT DIAGRAM

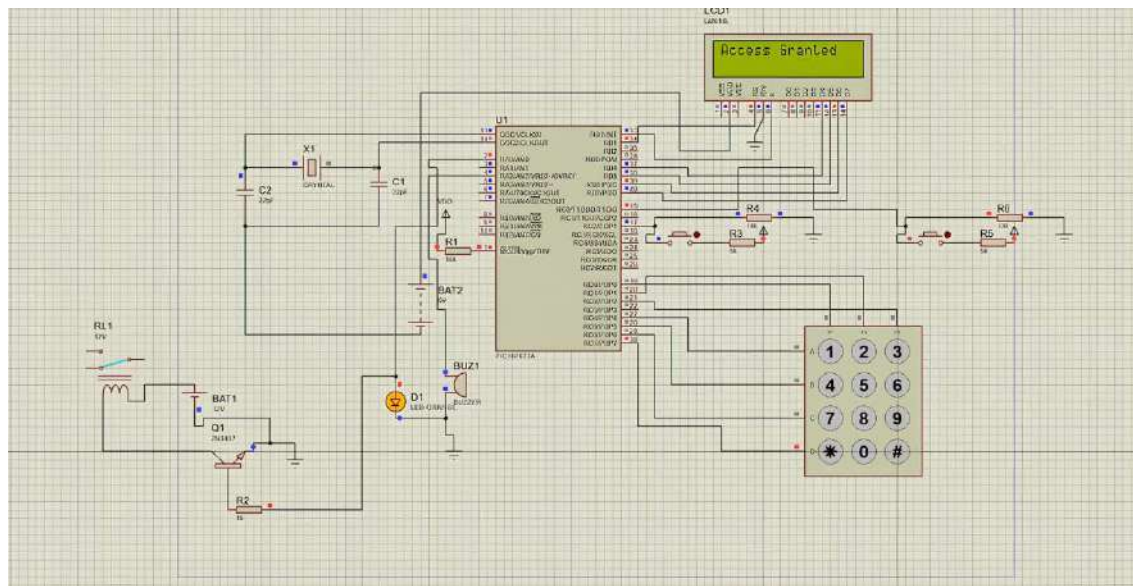


Figure: Microcontroller circuit simulation

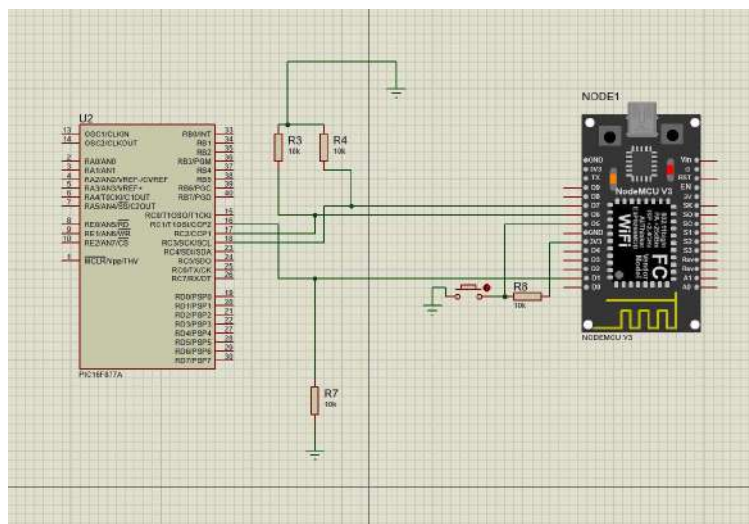


Figure: ESP8266 Wi-Fi module's connections with the pic microcontroller

3.4 SCHEMATIC DESIGN AND PCB LAYOUT

The schematic and layout design for the PCB was done using the EasyEDA software.

3.4.1 Schematic Design

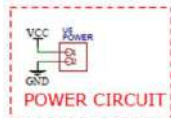


Figure: Power circuit

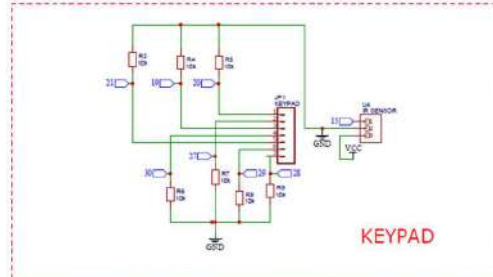


Figure: Keypad Circuit

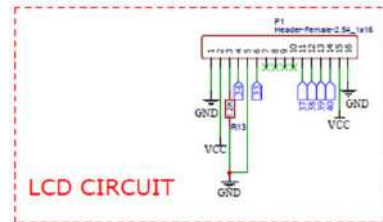


Figure: LCD Circuit

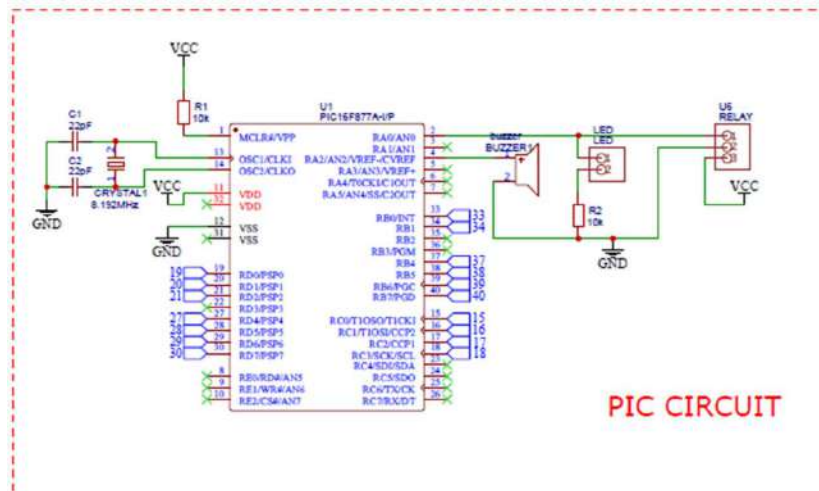


Figure: Microcontroller Circuit

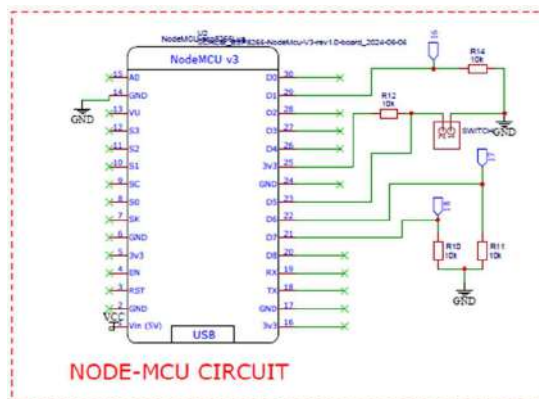


Figure: ESP8266 Wi-Fi Module Circuit

3.4.2 PCB Layout Design

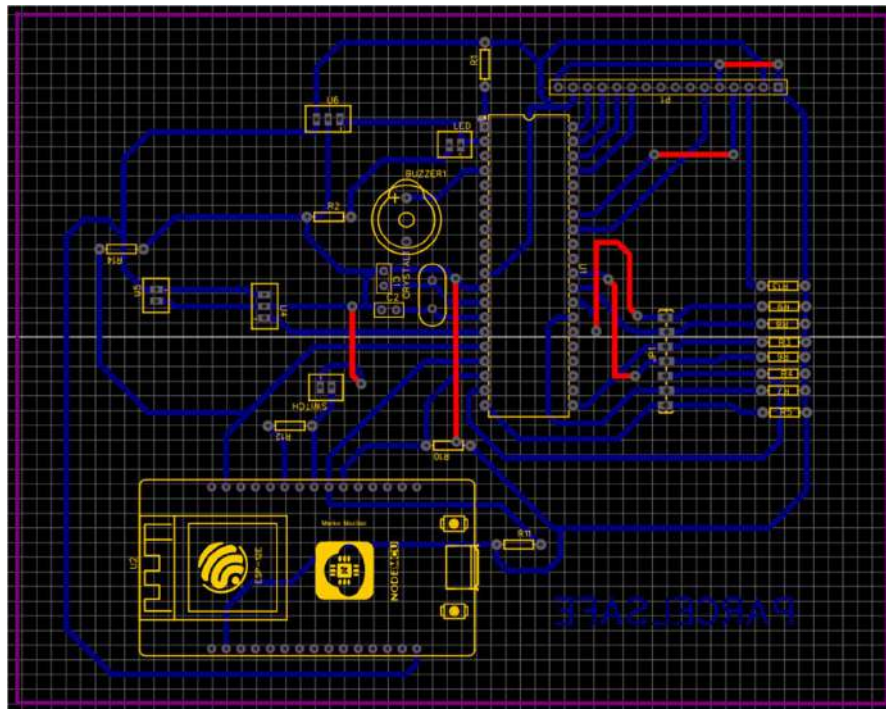


Figure: Top layer

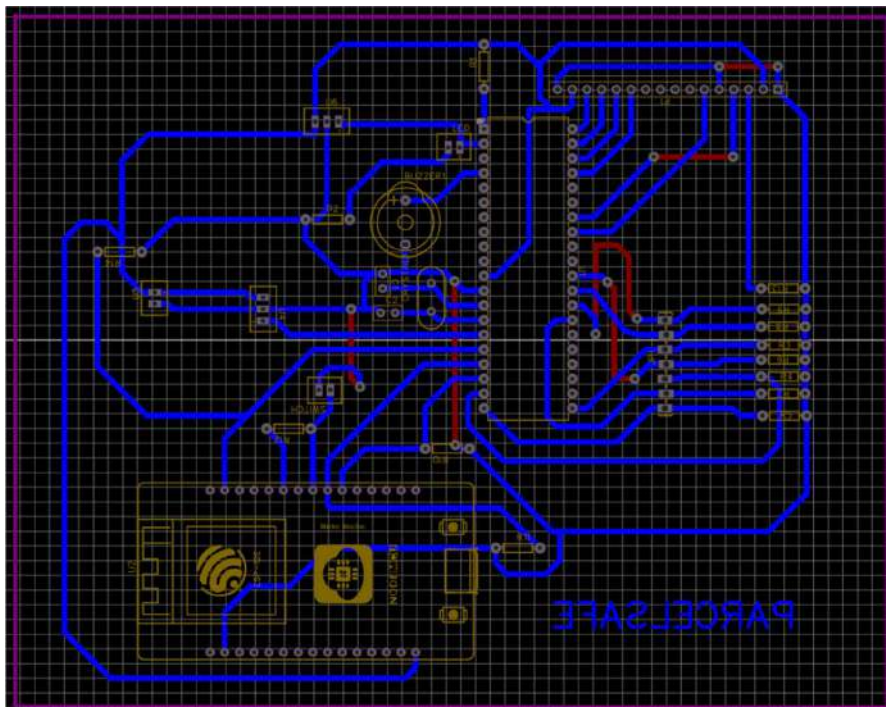


Figure: Bottom layer

The 6 connections which came to the top layer were connected to the bottom layer using vias, as there are no PCB manufacturers in Sri Lanka who prints double layer PCBs.

3.4.3 PCB Manufacturing

PCB manufacturing was done by a Sri Lankan PCB Manufacturer.

PCB Specifications

- Dimensions: 15 cm x 12 cm
- PCB thickness: 0.8mm

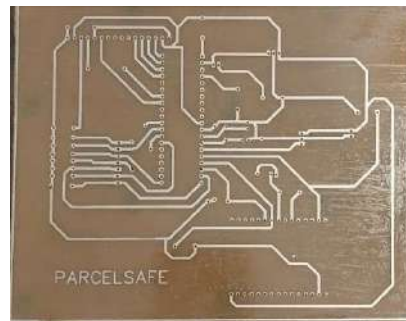


Figure: Manufactured PCB

3.5 ENCLOSURE DESIGN

3.5.1 SOLIDWORKS Design

Main Panel

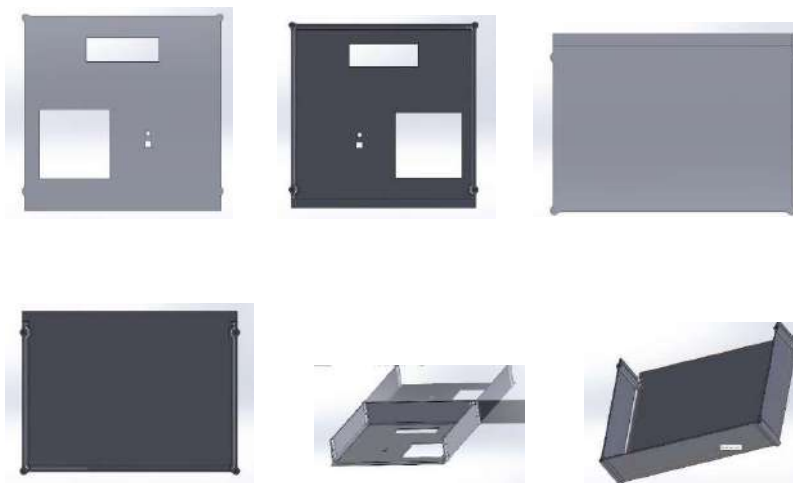


Figure: Panel (Main body)

Power circuit compartment



Figure: Compartment for power circuit

PCB after soldered and the Final Product

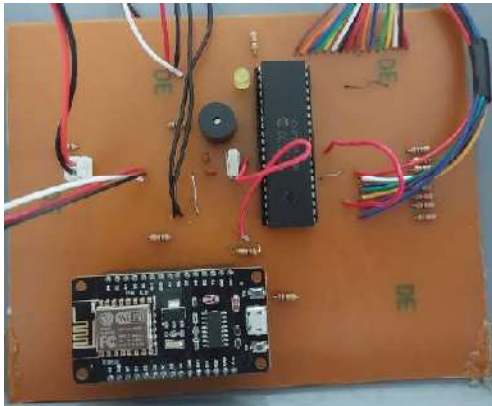


Figure: PCB after soldered



Figure: Final Product

3.5.2 Enclosure Manufacturing Methods

- The design of the box is based on an enclosure available in the market. This allows us to use this pre-made enclosure when manufacturing on the small scale in order to reduce costs.
- The panel containing electronic components was embedded into the door of the main box, and the compartment containing the power circuit was embedded into the side of the main box. These are designed as separate parts for easier manufacturing. For small scale manufacturing, this panel can also be manufactured by CNC machining a pre-made enclosure box and adding mounting spacers.
- For large scale manufacturing, both enclosures can be manufactured using injection molding. The design is made to meet injection molding requirements.
- For the prototype, a pre-made enclosure was used for the safe box and 3D printing was used for the remaining parts.

3.6 TOOLS, TECHNOLOGIES AND FRAMEWORKS EMPLOYED

3.6.1.1 Tools

- **Hardware components:**
 - PIC16F877A microcontroller – To program the device.
 - Two separately connected 3.7V and 2 boost converters – for power supply to PIC and lock.
 - IR sensor – to check the door opening.
 - ESP8266 Wi-Fi module – to connect the device to Wi-Fi.
- **Software components:**
 - mikroC Pro for PIC – to program the PIC microcontroller.
 - Arduino IDE – managing the Telegram bot and configuring the ESP8266.

3.6.1.2 Technologies

- **Programming languages:**
 - C programming language – for programming the microcontroller.
 - Arduino – for configuring ESP8266 and the Telegram bot.
- **Communication protocols:**
 - Wi-Fi – for enabling real time communication between the delivery locker and the recipient's Telegram App.
- **Notification System:**
 - Integration with the Telegram bot to notify users about the lock/unlock status of the delivery box.

3.6.1.3 Frameworks

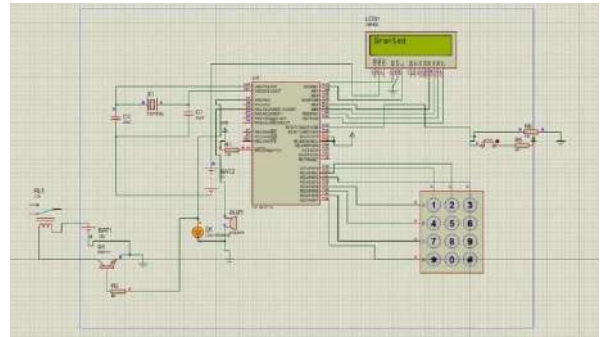
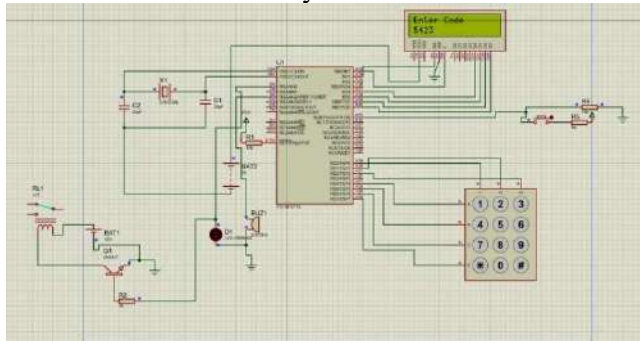
- **mikroC Pro for PIC Framework:**
Utilized for developing and managing the PIC's code with built-in libraries for ease of development.
- **Arduino Framework:** Used for configuring the ESP8266 and managing the Telegram bot.
UniversalTelegramBot Library was used for implementing the Telegram bot, which handles notifications and communication with the user's mobile phone.
- **EEPROM Library:** Employed for storing codes in the Wi-Fi module's memory, ensuring persistent data storage even after power cycles.

CHAPTER 04 : RESULTS AND ANALYSIS

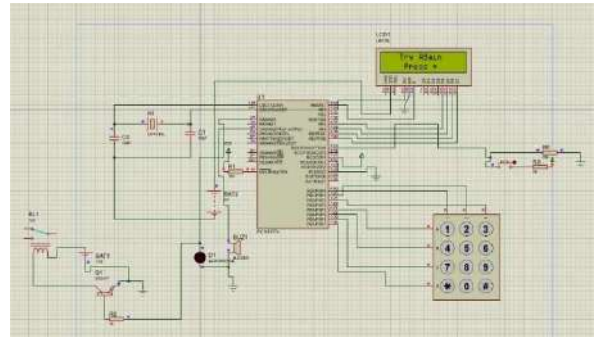
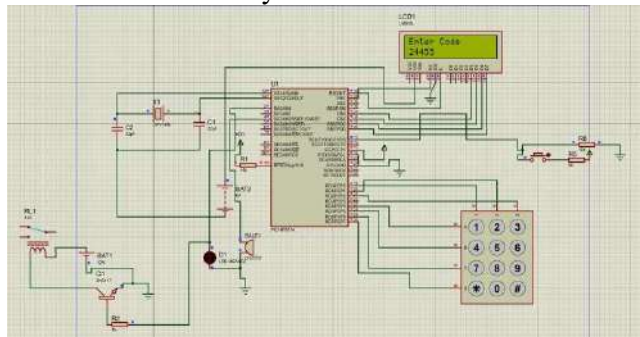
4.1 OUTCOMES AND RESULTS OF THE PROJECT

These are the results from the proteus simulation:

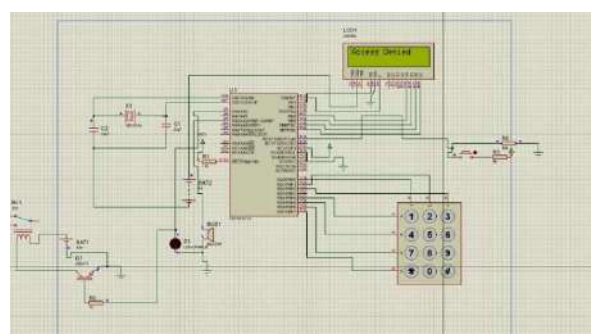
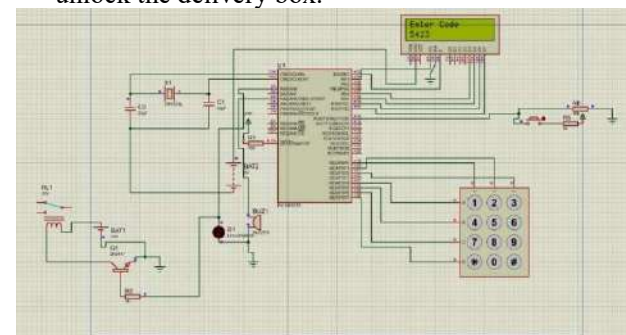
- When the delivery driver enters the correct code, and the user gives permission from the telegram bot to unlock the delivery box.



- When the delivery driver enters an incorrect code,



- When the delivery driver enters the correct code, the user doesn't give permission from the telegram bot to unlock the delivery box.



4.1.1 System Operation Workflow

- The user enters the master password provided in the manual, to set a delivery code.
- The delivery driver is provided with the delivery code and enters it upon delivery.
- The Telegram app notifies the user, who then grants or denies access to the delivery box.

- The IR sensor confirms the presence of a parcel.
 - The user specified Wi-Fi SSID and its corresponding password are stored in the EEPROM of the ESP8266.
1. **Initialization:** The LCD initializes and prompts the user to press the '*' key.
 2. **Code Entry:** The display then shows "Enter code". The delivery driver contacts the user to obtain this code and enters it, pressing '#' to submit.
 3. **Wi-Fi Configuration:** The LCD screen displays "Please wait..." until the Wi-Fi configuration is complete.
 4. **Wi-Fi Connection:** The user connects to the ESP8266's Wi-Fi network and navigates to the Wi-Fi configuration portal using any web browser.
 5. **Network Selection:** The user selects a Wi-Fi network, enters the SSID and password, and connects to the network.



```
*wm:Connecting to NEW AP: Siluni's Galaxy A52s 5G
*wm:connectTimeout not set, ESP waitForConnectResult...
*wm:Connect to new AP [SUCCESS]
*wm:Got IP Address:
*wm:192.168.25.67
*wm:config portal exiting
Connected to WiFi
WiFi credentials saved to EEPROM
Connected to WiFi SSID: Siluni's Galaxy A52s 5G
Detected high
```

6. **Notification and Approval:** Once the Wi-Fi is connected, a message is sent via Telegram asking whether to unlock the box, with two options: /YES and /NO.

- If /YES is given,



- If /NO is given,



4.2 COMPARISON OF THE RESULTS WITH THE EXISTING LITERATURE

When compared to existing solutions like parcel locker networks, Vpod Automated Parcel Lockers, and home secure parcel drop boxes, several key differences and improvements can be noted.

- **Security:**

Existing Solutions: Utilize access codes, QR codes, and facial recognition for security.

ParcelSafe: Enhances security with real-time user permissions via Telegram.

- **Convenience:**

Existing Solutions: Offer multiple authentication methods and 24/7 access to lockers.

ParcelSafe: Simplifies the process with a single code system, providing real-time notifications and easy user control, streamlining the user experience.

- **Technological Integration:**

Existing Solutions: Employ digital platforms for monitoring and management.

ParcelSafe: Integrates directly with Telegram for real-time user interaction.

4.3 WIFI CONNECTIVITY

In the ParcelSafe project, Wi-Fi connectivity, provided by the ESP8266 module configured with Arduino code, is crucial for real-time communication. It allows the system to send instant Telegram updates and alerts to users when the correct delivery code is entered, ensuring timely and secure package management.

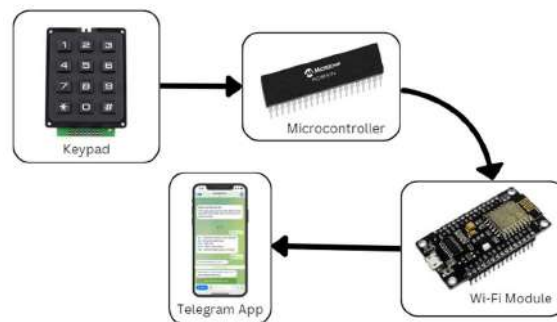


Figure: Connectivity

The ParcelSafe system operates as follows:

1. **Connect to the ESP8266's Network:** The user connects their device to the ESP8266's network, which opens a captive portal automatically.
2. **Configure Wi-Fi:** In the captive portal, the user selects "Configure Wi-Fi," chooses the desired network, enters the SSID and password, and presses "Save." A blinking blue LED indicates a successful connection.
3. **Entering Code and Sending Notifications:** When the correct code is entered on the keypad, the system sends a Telegram message asking whether to grant access to the box. The user replies with /YES to unlock the box or /NO to keep it locked.
4. **Resetting Wi-Fi:** The Wi-Fi reset button allows the user to reconfigure the Wi-Fi network settings as needed.

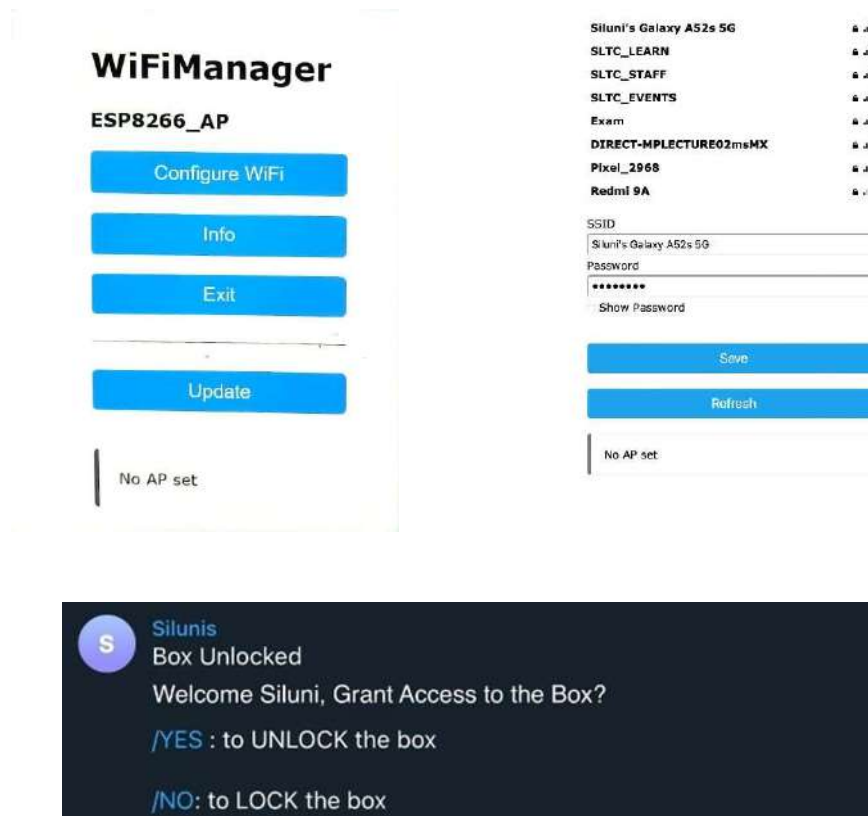


Figure: Wi-Fi settings interface and Telegram app's interface

CHAPTER 05 : CONCLUSION AND FUTURE WORK

5.1 FUTURE RESEARCH OR IMPROVEMENTS

1. Enhanced Power Management:

- Provide a battery charging module with a micro-USB cable, allowing users to charge the batteries inside the box without removing them.
- Connect the box power supply to an AC source from the house for continuous power.

2. Advanced Security Features:

- Improve the current single code system by generating random OTPs for each delivery driver to enhance security.
- Replace the IR sensor with a camera module to verify if the delivery person is placing the parcel inside and to monitor if they attempt to steal any existing contents.

3. Integrated Mobile App:

- Develop a custom app for ParcelSafe, named "ParcelSecure".

4. Multiple Enclosure Design Options:

- Provide different enclosure designs for various use cases, such as:
 - A small-sized box for small parcels.
 - A wall-mounted box powered by an AC source.
 - A design with a separate compartment for small documents.
 - Placing the components in the top section of the box, separated from user accessible compartment.

5. Emergency Access and Battery Management:

- Implement a method to open the door (e.g.: physical key) and place the parcel securely during a power outage or when the batteries are dead.
- Develop the app and system to provide battery status updates to the user, including warning messages when the battery is running low.

5.2 CONCLUSION

The ParcelSafe project successfully developed a prototype for a secure and automated delivery locker system, addressing the critical issue of unattended package deliveries. Through enhanced security measures, real-time notifications, and effective communication methods, ParcelSafe demonstrated its potential as a valuable solution for modern delivery challenges. Despite the encountered limitations, the project showcased the feasibility and benefits of such systems. Future research and improvements can further enhance ParcelSafe's functionality, reliability, and user experience, solidifying its relevance and impact in the domain of secure package delivery solutions.

5.3 WORKPLAN

5.3.1 Project Timeline



The tasks were,

Task 1: Finalizing and gathering components

Task 2: Creating the circuit

Task 3: Programming, testing, and troubleshooting

Task 4: Designing the PCB and soldering

Task 5: Integrating the PCB with the enclosure

5.3.2 Bill of Materials

Component	Quantity	Store	Unit Price (Rs.)
Solenoid Door Lock	1	Tronic.lk	1180
5V Relay Module	1	Tronic.lk	230
3.7V 1800mA Li-ion Battery	1	Tronic.lk	400
5V Mini Buzzer	1	Tronic.lk	30
4x3 Matrix Array Keypad switch module	1	Tronic.lk	750
IR Sensor	1	Tronic.lk	120
PIC Programming Adopter	1	Tronic.lk	700
NodeMCU ESP8266	1	Tronic.lk	800
Boost Converter	2	Tronic.lk	400
Rocker Switch	1	Tronic.lk	10
16-Way PCB mount wire clip with socket	1	Tronic.lk	110
Long 3-way PCB mount wire clip with socket	1	Tronic.lk	25
Short 7-way PCB mount wire clip with socket	1	Tronic.lk	50
Short 3-way PCB mount wire clip with socket	1	Tronic.lk	20
40-pin single row female headers	2	Tronic.lk	50

The price of the PIC microcontroller, LCD and one battery are removed as they were already bought for previous activities.

Rough cost of prototyping

Item	Cost
Electronic components	4875
PCB	650
3D printed enclosure parts	8950
Pre-made enclosure box	3500
Total	17975

5.3.3 Proposed Product Price

When manufacturing in bulk at the small business level, the enclosure cost can be reduced significantly by avoiding 3D printing and using pre-made enclosures with CNC machining. The effect of cost on one unit will be reduced when components and PCBs are ordered in bulk. The unit price may also be reduced.

Manufacturing cost considering the above factors

Item	Cost
Electronic components	3000
PCB	300
Pre-made enclosure box	5000
Total	8800

Keeping a profit margin of Rs. 2000, the product can be sold at a price of around Rs. 11000. This is significantly cheaper than similar products currently available on the market.

APPENDICES – PROGRAM CODE

PIC Microcontroller code using mikroC PRO for PIC

```
unsigned int seed;
unsigned short kp;
char code1[10], user1[4];
char msg1[20] = "Initializing.....", msg2[12] = "Enter Code";
char msg3[15] = "Access Granted", msg4[15] = "Access Denied";
char msg5[15] = "Enter New Code";
int i = 0, j, cnt, recVal = 0;
// Timer variable
volatile unsigned int elapsedTime = 0; // Time counter in milliseconds

void interrupt() {
    if (TMR1IF_bit) {
        TMR1IF_bit = 0; // Clear the Timer1 interrupt flag
        TMR1H = 0xFC;   // Reload Timer1 high byte for 1ms delay
        TMR1L = 0x18;   // Reload Timer1 low byte for 1ms delay
        elapsedTime++;  // Increment the elapsed time
    }
}

void initializeTimer1() {
    T1CON = 0x01; // Set Timer1 prescaler to 1:1
    TMR1H = 0xFC; // Load Timer1 high byte for 1ms delay
    TMR1L = 0x18; // Load Timer1 low byte for 1ms delay
    TMR1IE_bit = 1; // Enable Timer1 interrupt
    PEIE_bit = 1; // Enable peripheral interrupts
    GIE_bit = 1; // Enable global interrupts
    TMR1ON_bit = 1; // Turn on Timer1
}

void buzzer() {
    PORTA.B2 = 1;
    Delay_ms(200);
    PORTA.B2 = 0;
    Delay_ms(200);
    PORTA.B2 = 1;
    Delay_ms(200);
    PORTA.B2 = 0;
    Delay_ms(200);
    PORTA.B2 = 1;
    Delay_ms(200);
    PORTA.B2 = 0;
}
```

```

    Delay_ms(200);
}

// Keypad module connections
char keypadPort at PORTD;
// End Keypad module connections

// LCD module connections
sbit LCD_RS at RB1_bit;
sbit LCD_EN at RB0_bit;
sbit LCD_D4 at RB4_bit;
sbit LCD_D5 at RB5_bit;
sbit LCD_D6 at RB6_bit;
sbit LCD_D7 at RB7_bit;

sbit LCD_RS_Direction at TRISB1_bit;
sbit LCD_EN_Direction at TRISB0_bit;
sbit LCD_D4_Direction at TRISB4_bit;
sbit LCD_D5_Direction at TRISB5_bit;
sbit LCD_D6_Direction at TRISB6_bit;
sbit LCD_D7_Direction at TRISB7_bit;
// End LCD module connection

void code_enter() {
    kp = 0; // Reset key code variable
    // Wait for key to be pressed and released
    do
        kp = Keypad_Key_Click(); // Store key code in kp var
    while (!kp);

    // Prepare value for output, transform key to its ASCII value
    switch (kp) {
        case 1: kp = '1'; break; // 1
        case 2: kp = '2'; break; // 2
        case 3: kp = '3'; break; // 3
        case 5: kp = '4'; break; // 4
        case 6: kp = '5'; break; // 5
        case 7: kp = '6'; break; // 6
        case 9: kp = '7'; break; // 7
        case 10: kp = '8'; break; // 8
        case 11: kp = '9'; break; // 9
    }
}

```

```

        case 13: kp = 42; break; // *
        case 14: kp = 48; break; // 0
        case 15: kp = 35; break; // #
        default: break; // Default case to handle any unexpected key c
    }

    code1[i] = kp;
    Lcd_Chr(2, i + 1, kp);
    i++;
}

void delay_20ms() {
    Delay_ms(20);
}

void code_read() { // Read data from EEPROM
    delay_20ms();
    user1[0] = EEPROM_Read(0x00); // Read data from address 0
    delay_20ms();
    user1[1] = EEPROM_Read(0x01); // Read data from address 1
    delay_20ms();
    user1[2] = EEPROM_Read(0x02); // Read data from address 2
    delay_20ms();
    user1[3] = EEPROM_Read(0x03); // Read data from address 3
    delay_20ms();
}

void code_write() { // Write data to EEPROM
    delay_20ms();
    EEPROM_Write(0x00, code1[0]); // Write some data at a
    delay_20ms();
    EEPROM_Write(0x01, code1[1]); // Write some data at a
    delay_20ms();
    EEPROM_Write(0x02, code1[2]); // Write some data at a
    delay_20ms();
    EEPROM_Write(0x03, code1[3]); // Write some data at a
}

unsigned int pinValue; // IR sensor pin initialization

```



```

void change_code() {
    Lcd_Cmd(_LCD_CLEAR);           // Clear display
    Lcd_Out(1, 1, "Enter New Code");

    i = 0;
    code_enter();
    code_enter();
    code_enter();
    code_enter();
    code_write();
    code_read();
}

void main() {
    ADCON1 |= 0x07;               // Configure AN pins as digital
    TRISA = 0x00;
    TRISC.F0 = 1;                 // THIS IS FOR THE IR SENSOR
    TRISC.F1 = 0;                 // ESP COMMUNICATION PIN (PIC TO ESP)
    TRISC.F2 = 1;                 // ESP COMMUNICATION PIN (ESP TO PIC)
    TRISC.F3 = 1;                 // ESP COMMUNICATION PIN2 (ESP TO PIC)

    // Set direction to be output
    PORTA.B0 = 0;
    PORTA.B2 = 0;

    Keypad_Init();                // Initialize Keypad
    Lcd_Init();                   // Initialize Lcd

    code_read();

    // If no code is stored then default is 2345
    if (user1[0] == 0xFF && user1[1] == 0xFF && user1[2] == 0xFF && use
        EEPROM_Write(0x00, '2');           // Write some data at ad
        delay_20ms();
        EEPROM_Write(0x01, '3');           // Write some data at ad
        delay_20ms();
        EEPROM_Write(0x02, '4');           // Write some data at ad
        delay_20ms();
        EEPROM_Write(0x03, '5');           // Write some data at ad
    }
}

```

```

code_read();
Lcd_Cmd(_LCD_CLEAR);           // Clear display
Lcd_Cmd(_LCD_CURSOR_OFF);      // Cursor off
Lcd_Out(1, 1, msg1);
Delay_ms(500);
Lcd_Cmd(_LCD_CLEAR);           // Clear display

cnt = 0;

do {
    Lcd_Cmd(_LCD_CLEAR);
    PORTC.F1 = 0;
    Delay_ms(200);
    if (cnt < 1) {
        i=0;
        delay_ms(500);
        Lcd_Out(2, 3, "Press *");
        Delay_ms(200);
    } else {
        i=0;
        delay_ms(500);
        Lcd_out(1, 4, "Try Again ");
        Lcd_out(2, 5, "Press *");
        Delay_ms(200);
    }

    i = 0;
    code_enter();
    if (code1[0] == 42) {
        Lcd_Cmd(_LCD_CLEAR);           // Clear display
        Lcd_Out(1, 1, msg2);
        i = 0;
        code_enter();
        code_enter();
        code_enter();
        code_enter();
        code_enter();

        // CHANGE MASTER CODE
        if (code1[0] == '2' &&
            code1[1] == '4' &&

```

```

code1[2] == '4' &&
code1[3] == '5' &&
code1[4] == '5') { // Check master code
    code_enter();
    if (code1[5] == 35) {
        change_code();
        cnt = 0;

        Lcd_Cmd(_LCD_CLEAR);
        Lcd_Out(1, 3, "Code Changed");
        Delay_ms(1500);
        continue;
    }
}

if (cnt < 3 &&
code1[0] == user1[0] &&
code1[1] == user1[1] &&
code1[2] == user1[2] &&
code1[3] == user1[3] &&
code1[4] == 35) { // Compare code with stored one
    cnt = 0;
    PORTC.F1 = 1; //Turn ESP pin active high
    PORTA.BO = 0;
    recVal = 1;
    Lcd_Cmd(_LCD_CLEAR);
    Lcd_Out(1, 1, "Please wait");
    Delay_ms(800);

    1 = 0;

} else {
    cnt++;
    Lcd_Cmd(_LCD_CLEAR);
    Lcd_Out(1, 1, msg4);
}

while (recVal == 1) {

```

```

if (PORTC.F2 == 1 && PORTC.F3 == 1) {
    PORTC.F1=0;

    Lcd_Cmd(_LCD_CLEAR);
    Delay_ms(50);
    Lcd_Out(1, 1, "Granted");
    PORTA.BO = 1;

    recVal = 0;
    buzzer();
    Delay_ms(100);
    initializeTimer1();
    Delay_ms(1000);
    elapsedTime = 0;

    while (PORTC.F0 == 1 && elapsedTime < 20000){

        PORTA.BO = 1;
        pinValue = PORTC.F0;

    }

    PORTB.F0 = 0;
    TMR1ON_bit = 0;
    PORTA.BO = 0;
}

Delay_ms(100);
if (PORTC.F2 == 1 && PORTC.F3 == 0) {
    PORTC.F1=0;
    PORTA.BO = 0;
    Lcd_Cmd(_LCD_CLEAR);
    Delay_ms(2000);
    Lcd_Out(1, 1, "Access Denied");

```

```

        Delay_ms(2000);
        recVal = 0;
    }
}

if (cnt >= 3) {
    do {
        PORTA.B2 = 1;
    } while (1);
}

) while (1);
}

```

ESP8266 Wi-Fi Module's code using Arduino IDE

```
1  #include <ESP8266WiFi.h>
2  #include <EEPROM.h>
3  #include <WiFiClientSecure.h>
4  #include <UniversalTelegramBot.h>
5  #include <WiFiManager.h>
6
7  #define SWITCH_PIN D5           // GPIO0 is usually connected to a button (adjust as necessary)
8  #define LED_PIN 2               // GPIO2 for the built-in LED on most ESP8266 boards
9  #define EEPROM_SIZE 64
10
11 #define picToEsp D1 //this goes to PORTC.F1 which is RC1
12
13 #define espToPic D6 //This goes to PORTC.F2 which is RC2
14 #define espToPic2 D7 //This goes to PORTC.F3 Which is RC3
15
16 #define BOT_TOKEN "7494735194:AAHck4J6GphTRzZKF7Y0chH3h7qucQVLCQI" // Initialize Telegram token
17
18 const unsigned long BOT_MTBS = 200; // Mean time between scan messages
19
20 WiFiClientSecure secured_client;
21 UniversalTelegramBot bot(BOT_TOKEN, secured_client);
22 unsigned long bot_lasttime;
23 int ledStatus = 0;
24
25 WiFiManager wifiManager;
26
27 void setup() {
28     Serial.begin(9600);
29     pinMode(SWITCH_PIN, INPUT_PULLUP);
30     pinMode(LED_PIN, OUTPUT);
31     pinMode(picToEsp, INPUT);
32     pinMode(espToPic, OUTPUT);
33     pinMode(espToPic2, OUTPUT);
34
35     digitalWrite(LED_PIN, LOW); // Turn off LED initially
```

```

36
37 // Initialize EEPROM
38 EEPROM.begin(EEPROM_SIZE);
39
40 // Check if we need to configure WiFi
41 bool configWiFi = digitalRead(SWITCH_PIN) == LOW || !loadWiFiCredentials();
42
43 if (configWiFi) {
44     Serial.println("Starting WiFi configuration...");
45     startConfigPortal();
46 }
47
48 Serial.print("Connected to WiFi SSID: ");
49 secured_client.setInsecure(); // Disable certificate validation (use with caution)
50 Serial.println(WiFi.SSID()); // Print the connected WiFi SSID
51 blinkLED(2); // Blink LED twice to indicate successful connection
52 }
53
54 void loop() {
55     digitalWrite(espToPic, 0);
56     digitalWrite(espToPic2, 0);
57     // Check if the switch is pressed
58     if (digitalRead(SWITCH_PIN) == LOW) {
59         Serial.println("Switch pressed. Starting WiFi configuration...");
60         startConfigPortal();
61         while(digitalRead(SWITCH_PIN) == LOW){
62
63         }
64         blinkLED(2); // Blink LED twice to indicate successful reconfiguration
65     }
66
67     if (digitalRead(picToEsp) == HIGH) {
68         Serial.println("Detected high");
69         String chat_id = "848564756"; // Replace with your chat ID
70         String welcome = "Welcome Ms.Pasqual ,Grant access to the box?. \n";

```

```

71
72 welcome += "/YES : to UNLOCK the box\n\n";
73 welcome += "/NO: to LOCK the box \n";
74
75 bot.sendMessage(chat_id, welcome, "Markdown");
76
77 // Wait until picToEsp pin goes low
78 while (digitalRead(picToEsp) == HIGH) {
79     if (millis() - bot_lasttime > BOT_MTBS) {
80         int numNewMessages = bot.getUpdates(bot.last_message_received + 1);
81
82         while (numNewMessages) {
83             Serial.println("got response");
84             for (int i = 0; i < numNewMessages; i++) {
85                 String text = bot.messages[i].text;
86                 if (text == "/YES") {
87                     bot.sendMessage(chat_id, "Box Unlocked", "");
88                     digitalWrite(espToPic, HIGH);
89                     digitalWrite(espToPic2, HIGH);
90                     delay(20);
91                     digitalWrite(LED_PIN, LOW); // turn the LED on (HIGH is the voltage level)
92
93                 } else if (text == "/NO") {
94                     bot.sendMessage(chat_id, "Box Locked", "");
95                     digitalWrite(espToPic, HIGH);
96                     digitalWrite(espToPic2, LOW);
97                     digitalWrite(LED_PIN, HIGH); // turn the LED off (LOW is the voltage level)
98
99                 }
100             }
101             numNewMessages = bot.getUpdates(bot.last_message_received + 1);
102         }
103         bot_lasttime = millis();
104     }
105 }
106 }

```



```
107
108 // Your main code here
109 if (millis() - bot_lasttime > BOT_MTBS) {
110     int numNewMessages = bot.getUpdates(bot.last_message_received + 1);
111
112     while (numNewMessages) {
113         Serial.println("got response");
114         handleNewMessages(numNewMessages);
115         numNewMessages = bot.getUpdates(bot.last_message_received + 1);
116     }
117
118     bot_lasttime = millis();
119 }
120
121
122 }
123
124 bool loadWiFiCredentials() {
125     // Check if SSID and password are stored in EEPROM
126     char ssid[32];
127     char password[32];
128
129     for (int i = 0; i < 32; ++i) {
130         ssid[i] = EEPROM.read(i);
131         password[i] = EEPROM.read(32 + i);
132     }
133
134     if (ssid[0] == '\0' || password[0] == '\0') {
135         Serial.println("WiFi credentials not found in EEPROM");
136         return false;
137     }
138 }
```

```

139   WiFi.begin(ssid, password);
140
141   // Wait for connection
142   int attempts = 10;
143   while (WiFi.status() != WL_CONNECTED && attempts > 0) {
144       delay(1000);
145       Serial.print(".");
146       attempts--;
147
148       // Check if the switch is pressed during connection attempts
149       if (digitalRead(SWITCH_PIN) == LOW) {
150           Serial.println("\nSwitch pressed during connection attempt. Clearing EEPROM and resetting.");
151           clearEEPROM();
152           ESP.restart();
153       }
154   }
155
156   if (WiFi.status() != WL_CONNECTED) {
157       Serial.println("Failed to connect to WiFi");
158       return false;
159   }
160
161   Serial.print("Connected to WiFi SSID: ");
162   Serial.println(WiFi.SSID()); // Print the connected WiFi SSID
163
164   return true;
165 }
166
167 void saveWiFiCredentials() {
168     // Get SSID and password from WiFiManager
169     String ssid = WiFi.SSID();
170     String password = WiFi.psk();
171
172     // Save SSID and password to EEPROM
173     for (int i = 0; i < 32; ++i) {
174         EEPROM.write(i, i < ssid.length() ? ssid[i] : '\0');
175         EEPROM.write(32 + i, i < password.length() ? password[i] : '\0');

```

```

176     }
177
178     EEPROM.commit();
179     Serial.println("WiFi credentials saved to EEPROM");
180 }
181
182 void startConfigPortal() {
183     // Reset settings - for testing
184     wifiManager.resetSettings();
185
186     // Set a custom timeout for the configuration portal
187     wifiManager.setConfigPortalTimeout(180);
188
189     // Start the config portal without a password
190     bool portalRunning = true;
191     while (portalRunning) {
192         // Check if the switch is pressed during the portal running
193         if (digitalRead(SWITCH_PIN) == LOW) {
194             Serial.println("\nSwitch pressed during configuration portal. Clearing EEPROM and resetting.");
195             clearEEPROM();
196             ESP.restart();
197         }
198
199         // Start config portal and check for timeout or connection
200         if (wifiManager.autoConnect("Ms Pasqual's")) {
201             Serial.println("Connected to WiFi");
202             saveWiFiCredentials();
203             portalRunning = false;
204         } else {
205             Serial.println("Failed to connect and hit timeout");
206             delay(3000);
207             ESP.restart();
208         }
209     }
210 }
211

```

```

212 void clearEEPROM() {
213     for (int i = 0; i < EEPROM_SIZE; ++i) {
214         EEPROM.write(i, 0);
215     }
216     EEPROM.commit();
217     Serial.println("EEPROM cleared");
218 }
219
220 void blinkLED(int times) {
221     for (int i = 0; i < times; i++) {
222         digitalWrite(LED_PIN, HIGH); // Turn on LED
223         delay(200);                  // Wait for 200 ms
224         digitalWrite(LED_PIN, LOW);  // Turn off LED
225         delay(200);                  // Wait for 200 ms
226     }
227 }
228
229 void handleNewMessages(int numNewMessages) {
230     Serial.print("handleNewMessages ");
231     Serial.println(numNewMessages);
232
233     for (int i = 0; i < numNewMessages; i++) {
234         String chat_id = bot.messages[i].chat_id;
235         String text = bot.messages[i].text;
236
237         String from_name = bot.messages[i].from_name;
238         if (from_name == "") from_name = "Guest";
239
240         if (text == "/ledon") {
241             digitalWrite(LED_PIN, 0); // turn the LED on (HIGH is the voltage level)
242             ledStatus = 1;
243             bot.sendMessage(chat_id, "Led is ON", "");
244         }
245
246         if (text == "/ledoff") {
247             ledStatus = 0;
248             digitalWrite(LED_PIN, 1); // turn the LED off (LOW is the voltage level)
249             bot.sendMessage(chat_id, "Led is OFF", "");
250         }
251
252         if (text == "/status") {
253             if (ledStatus) {
254                 bot.sendMessage(chat_id, "Led is ON", "");
255             } else {
256                 bot.sendMessage(chat_id, "Led is OFF", "");
257             }
258         }
259
260         if (text == "/start") {
261             String welcome = "Welcome Ms.Pasqual to Universal Arduino Telegram Bot library. \n";
262             welcome += "This is Flash Led Bot example.\n\n";
263             welcome += "/ledon : to switch the Led ON\n";
264             welcome += "/ledoff : to switch the Led OFF\n";
265             welcome += "/status : Returns current status of LED\n";
266             bot.sendMessage(chat_id, welcome, "Markdown");
267         }
268     }
269 }

```

REFERENCES

[1] “PIC16F87XA Data Sheet 28/40/44-Pin Enhanced Flash Microcontrollers.”

Available: <https://ww1.microchip.com/downloads/en/devicedoc/39582b.pdf>

[2] “ESP8266EX Datasheet.” Available:

https://www.espressif.com/sites/default/files/documentation/0a-esp8266ex_datasheet_en.pdf

[3] “Interfacing 4x3 Membrane Matrix Keypad with Arduino,” *Electropeak*, Feb. 13, 2021.

<https://electropeak.com/learn/interfacing-4x3-membrane-matrix-keypad-with-arduino/>

[4] V. K. M, “vishaletm/Pic-16F877A-Based-Code-Lock-16x2-LCD-Keypad,” *GitHub*, Mar.

25, 2024. <https://github.com/vishaletm/Pic-16F877A-Based-Code-Lock-16x2-LCD-Keypad>

[5] B. Lough, “Universal Telegram Bot Library,” *GitHub*, Nov. 11, 2023.

<https://github.com/witnessmenow/Universal-Arduino-Telegram-Bot/tree/master>

[6] M. Technology, “IR Sensor interfacing with microcontroller (Mikro C),” *MINA*

TECHNOLOGY. <https://minatechnologysupport.blogspot.com/2019/08/ir-sensor-interfacing-with.html>